

KUBERNETES NOTES

Your Handy Guide to Kubernetes Concepts

A comprehensive collection of Kubernetes concepts, explained in simple words with clear diagrams and examples. Perfect for quick revision and interview preparation.

What's Inside?

- Core Concepts
- Architecture
- Pods & Controllers
- Networking
- Storage
- Deployments
- Best Practices

Why These Notes?

- Easy to understand
- Well-structured
- Interview focused
- Quick revision
- Handwritten style for better retention

"Learn, Deploy, and Scale with Kubernetes!"

KUBERNETES NOTES

— Page 1 —

1. What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform. It helps us automate the deployment, scaling, management, and networking of containers. It was originally developed by Google and is now maintained by CNCF.

In Simple Words

Kubernetes helps us run our containerized applications reliably at scale.

2. Why Kubernetes was Created?

Earlier we were using Docker to run containers. But managing multiple containers manually was very difficult. Kubernetes solves these problems.

Problems with Docker Alone

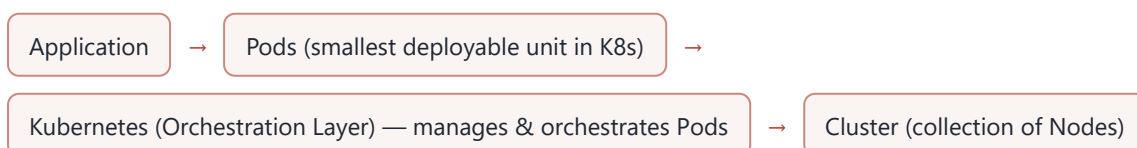
- Hard to manage multiple containers
- No auto scaling
- No self-healing if a container fails
- No load balancing
- No rolling updates
- Difficult to manage networking and storage

3. Features of Kubernetes

- Automatic deployment and scaling
- Self-healing (restarts failed containers)
- Load balancing and service discovery
- Rolling updates and rollback
- Storage orchestration
- Secret and configuration management
- Runs on any environment (On-prem, Cloud, Hybrid)

4. Container Orchestration

Container orchestration means managing the entire lifecycle of containers like deploying, scaling, networking, monitoring, etc. automatically.



Docker vs Kubernetes

Docker	Kubernetes
Manages single container	Manages multiple containers
Manual process	Automatic process
No scaling, self-healing	Auto scaling, self-healing

KUBERNETES NOTES

— Page 2 —

1. Kubernetes Architecture

Kubernetes follows a Master-Worker (Client-Server) architecture. The Cluster is divided into two main parts:

- Control Plane (Master)
- Worker Nodes



2. Control Plane Components

Control Plane makes global decisions about the cluster (e.g. scheduling), detects and responds to cluster events, and maintains the desired state.

Component	Role
API Server	Frontend of Kubernetes. Exposes the REST API used by kubectl and other components. All requests go through it.
etcd	Consistent and highly-available key-value store for all cluster data.
Scheduler	Watches for newly created Pods and selects a Node for them to run on.
Controller Manager	Runs controller processes (Node Controller, ReplicaSet Controller, etc.) that regulate the state of the cluster.

3. Worker Node Components

Worker Nodes run the containerized applications and maintain the running environment.

Component	Role
kubelet	Communicates with Control Plane. Ensures containers are running in a Pod and manages Pods/containers.
kube-proxy	Handles networking and service communication. Maintains network rules.
Container Runtime (containerd / Docker)	Pulls images and runs the containers.

In short: Control Plane manages the Cluster, while Worker Nodes run the applications.

KUBERNETES NOTES

— Page 3 —

1. What is a Pod?

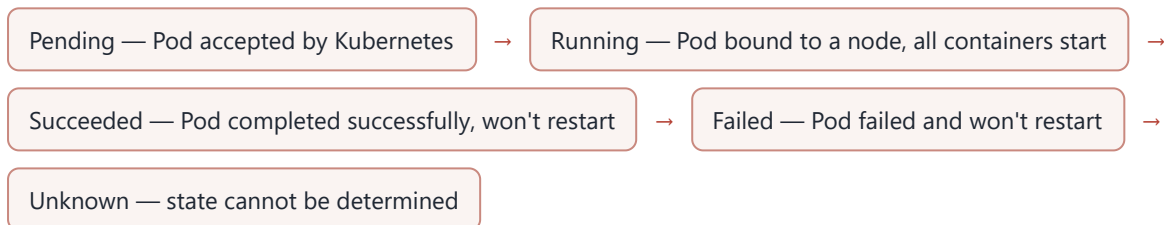
A Pod is the smallest deployable unit in Kubernetes. It represents one or more containers that share the same network, storage, and configuration. Containers in a Pod are always scheduled together on the same node and share network and storage resources.

2. Pod Types

- **Single Container Pod** — Contains only one container.
- **Multi Container Pod** — Contains multiple containers that work closely together.

3. Pod Lifecycle

A Pod goes through different phases during its lifecycle.



Pod may restart and go back to Pending or Running state.

4. Why Pod instead of Container?

- Pods are the basic building block in K8s.
- Allows multiple containers to work together.
- Shares network, storage, and config.
- Provides better resource management.
- Easier scaling and self-healing.

Container (Docker)	Pod (Kubernetes)
Single container	One or more containers
Manual management	Managed by Kubernetes
No self-healing	Self-healing (restarts on failure)
Scaling is manual	Auto scaling is possible
No shared resources	Shares network & storage

In short: Pod is the smallest, simplest unit in Kubernetes that Kubernetes creates or deploys. It wraps one or more containers that are tightly coupled.

KUBERNETES NOTES

— Page 4 —

1. Deployments

A Deployment manages ReplicaSets and provides declarative updates for Pods and ReplicaSets. It helps in: Scaling, Rolling Updates, Rollback, Self-healing.

Deployment (manages ReplicaSet) → ReplicaSet → Pods (runs desired number of Pods)

2. ReplicaSet

ReplicaSet ensures that a specified number of Pod replicas are running at any time. If a Pod fails, ReplicaSet creates a new one to maintain the desired state.

Key Points

- Ensures high availability
- Maintains the desired number of Pods
- Monitors Pods continuously
- Works under Deployment

3. Scaling

We can scale the number of Pods up or down as per the load.

- **Manual Scaling:** Using `kubectl scale`
- **Auto Scaling:** Using HPA (Horizontal Pod Autoscaler)

4. Rolling Update

Rolling Update allows us to update the application without downtime. New Pods are created one by one and old Pods are terminated gradually.

Old Pod Running → New Pod Created → Old Pod Terminated → New Pod Running

5. Rollback

If something goes wrong after an update, we can rollback to the previous stable version easily.

6. kubectl Commands (Deployments)

Command	Description	Example
<code>kubectl get deployments</code>	List all deployments	<code>kubectl get deployments</code>
<code>kubectl create deployment <name> -- image=<image></code>	Create a new deployment	<code>kubectl create deployment myapp -- image=nginx</code>

<code>kubectl scale deployment <name> --replicas=<n></code>	Scale deployment to n replicas	<code>kubectl scale deployment myapp --replicas=5</code>
<code>kubectl rollout status deployment/<name></code>	Check rollout status	<code>kubectl rollout status deployment/myapp</code>
<code>kubectl rollout undo deployment/<name></code>	Rollback to previous version	<code>kubectl rollout undo deployment/myapp</code>
<code>kubectl set image deployment/<name> <container>=<image></code>	Update image of container	<code>kubectl set image deployment/myapp nginx=nginx:1.25</code>

KUBERNETES NOTES

— Page 5 —

1. Services — Why?

Pods are ephemeral (can be created, destroyed, recreated). Their IPs keep changing. A Service provides a stable IP and DNS name to access Pods. It also helps in load balancing and distributes traffic to Pods.

2. Service Types

Type	Use	Accessible From	Description
ClusterIP	Internal communication inside cluster	Inside the cluster	Default type. Exposes the Service on a cluster-internal IP.
NodePort	Expose service on each Node's IP	Outside the cluster (via Node IP : Port)	Exposes the Service on a static port on each Node's IP.
LoadBalancer	Expose service to external traffic	Outside the cluster (Cloud Load Balancer)	Uses cloud provider's load balancer to expose the Service.
ExternalName	Map service to an external domain	Inside the cluster	Maps the Service to a DNS name outside the cluster.

3. Service Types — Diagrams

- **ClusterIP:** Client (inside cluster) → Service (ClusterIP) → Pods. Accessible only inside the cluster.
- **NodePort:** Client (outside cluster) → Node IP : NodePort → Service (NodePort) → Pods. Accessible from outside via Node IP and NodePort.
- **LoadBalancer:** Client (outside cluster) → Load Balancer (Cloud) → Service (LoadBalancer) → Pods. Accessible from outside via Cloud Load Balancer.
- **ExternalName:** Client (inside cluster) → Service (ExternalName) → external-domain.com. Maps to external DNS name.

4. Service Example (ClusterIP)

- Service Name: my-service
- Type: ClusterIP
- Cluster IP: 10.96.0.15
- Port: 80
- Target Port: 8080

Client inside cluster accesses `my-service:80`. Traffic is load balanced across Pods.

KUBERNETES NOTES

— Page 6 —

1. ConfigMap — Why?

ConfigMap is used to store non-sensitive configuration data in key-value pairs. It allows us to separate configuration from the container image. Stores configuration data as key-value pairs (e.g. app.properties, db.url, log.level).

2. ConfigMap — Use Cases

- Application configuration
- Environment variables
- Command line arguments
- Configuration files

3. Secret — Why?

Secret is used to store sensitive data like passwords, tokens, keys, etc. It is Base64 encoded (not encrypted) and more secure than ConfigMap. Stores sensitive data as key-value pairs (e.g. username, password, token).

4. Secret — Use Cases

- Passwords
- API Keys
- OAuth Tokens
- TLS Certificates

5. ConfigMap vs Secret

Feature	ConfigMap	Secret
Data Type	Non-sensitive	Sensitive
Encoding	Plain Text	Base64 Encoded
Purpose	Configuration	Secrets (passwords, keys)
Security	Less Secure	More Secure
Example	app.properties	db-password

6. How ConfigMap & Secret are used in Pod

A Pod consumes ConfigMap and Secret data as Environment Variables or as files via Volume Mount.

7. Important Points

- ConfigMap and Secret help in decoupling configuration from application.

- Secrets are not encrypted by default in etcd. Enable encryption for better security.
- Both can be used as Environment Variables or mounted as Volume inside Pods.

KUBERNETES NOTES

— Page 7 —

1. Volumes — Why?

Containers are ephemeral. When a container is deleted, all data inside it is lost. Volumes provide persistent storage that survives container restarts and deletions. Data persists even if the container is deleted.

2. Volume Types

Type	Description
emptyDir	Temporary storage, deleted when Pod is removed.
hostPath	Mounts a file or directory from the host node.
nfs	Mounts NFS share.
configMap	Mounts ConfigMap as files.
secret	Mounts Secret as files.
persistentVolumeClaim	Uses persistent storage (PV).

Notes

Use emptyDir for temporary data. Use hostPath with caution. Use PVC for production workloads.

3. Persistent Storage — PV and PVC

- **PV (Persistent Volume):** A piece of storage in the cluster provisioned by admin or dynamically. Actual storage in the cluster.
- **PVC (Persistent Volume Claim):** A request for storage by users. It binds to a suitable PV.



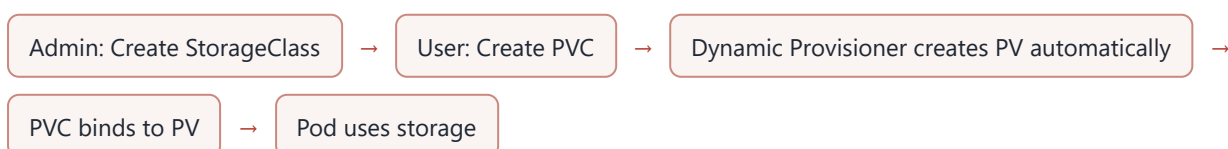
4. Storage Class

StorageClass provides a way to describe the "class" of storage you want. It enables dynamic provisioning of PVs.

Example

fast-ssd : For high performance · standard : For general purpose · slow-hdd : For backup, logs

5. Storage Flow



If no StorageClass is defined, static PV needs to be created manually. Dynamic provisioning is recommended.

6. Useful Commands (Volumes)

Command	Description
<code>kubectl get pv</code>	List all Persistent Volumes
<code>kubectl get pvc</code>	List all Persistent Volume Claims
<code>kubectl describe pvc <name></code>	Describe a PVC
<code>kubectl delete pvc <name></code>	Delete a PVC
<code>kubectl get storageclass</code>	List all Storage Classes
<code>kubectl describe storageclass <name></code>	Describe a Storage Class

KUBERNETES NOTES

— Page 8 —

1. Namespaces

Namespace provides a way to divide cluster resources between multiple users, teams, or environments.

- Default namespace is "default".
- Helps in isolation and resource management.
- Resources in one namespace are not visible in another.

Note

Resources in different namespaces are isolated from each other. A pod in namespace 1 cannot access a pod in namespace 2 directly.

2. Ingress — Why?

Service of type LoadBalancer/NodePort exposes applications, but Ingress provides more powerful routing (host based / path based) using a single entry point.

3. Ingress — Key Points

- Works at Layer 7 (Application Layer).
- Supports host based routing.
- Supports path based routing.
- TLS/SSL termination.
- Works with Ingress Controller.

An Ingress Controller (e.g. Nginx, Traefik) implements Ingress rules and routes traffic to services.

4. Ingress Example (Host & Path based Routing)

Example Rules

Host: app.example.com
Path: /api → Service : api-service
Path: /web → Service : web-service
Host: admin.example.com
Path: / → Service : admin-service

5. Service vs Ingress

Feature	Service	Ingress
Layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP/HTTPS)
Routing	Basic (Port based)	Advanced (Host / Path based)

SSL/TLS	Not handled	Can handle SSL termination
Usage	Expose service	Expose multiple services via rules
Controller	Not required	Requires Ingress Controller

6. Useful Commands

Command	Description
<code>kubectl get ns</code>	List all namespaces
<code>kubectl get ingress -A</code>	List all Ingress in all namespaces
<code>kubectl describe ingress <name></code>	Describe Ingress
<code>kubectl apply -f ingress.yaml</code>	Create Ingress from YAML
<code>kubectl delete ingress <name></code>	Delete Ingress

In short: Namespaces help in isolation. Ingress helps in smart routing of traffic to services.

KUBERNETES NOTES

— Page 9 —

1. Deployments — Why?

- Deployments provide declarative updates for Pods and ReplicaSets.
- They help in managing application lifecycle.
- Supports rolling updates and rollbacks.
- Self-healing: ensures desired number of Pods are running.
- Acts as a higher-level abstraction over ReplicaSet.

Deployment (manages ReplicaSet — Desired State)

→

ReplicaSet (manages Pods)

→

Pods (running containers)

2. ReplicaSet vs Deployment

Feature	ReplicaSet	Deployment
Purpose	Ensures a specified number of Pod replicas are running.	Manages ReplicaSets and provides declarative updates.
Updates	No rolling updates.	Supports rolling updates.
Rollbacks	Not supported.	Supports rollbacks to previous versions.
Scaling	Supported (Replicas).	Supported (Replicas).
Self-healing	Yes, maintains desired state.	Yes, maintains desired state.
Use Case	Use directly in simple use cases.	Use in production for managing applications.

3. Rolling Update — How it Works?

Step 1: Current State — All Pods running old version (v1)

→

Step 2: New ReplicaSet (v2) created with new Pods

→

Step 3: Old Pods terminated one by one, new Pods come up

→

Step 4: Update Complete — all Pods running new version (v2)

4. Rollback — How it Works?

Deployment Update Fails

→

`kubectl rollout undo deployment <name>`

→

Deployment reverts to previous version

→ Pods updated back to stable version

Example Command

```
kubectl rollout undo deployment my-app · kubectl rollout history deployment my-app
```

5. Deployment Strategies

- **Recreate** (Default for some resources): All old Pods are terminated first, new Pods are created after.
- **RollingUpdate** (Default for Deployments): Pods are updated one by one. Zero downtime.
- **Blue/Green Deployment** (Advanced): Two environments (Blue & Green). Switch traffic to new version.

6. Useful Commands (Deployments)

Command	Description	Example
<code>kubectl create deployment <name> --image=<image></code>	Create a new Deployment	<code>kubectl create deployment myapp --image=nginx</code>
<code>kubectl get deployments</code>	List all Deployments	<code>kubectl get deployments</code>
<code>kubectl get rs</code>	List all ReplicaSets	<code>kubectl get rs</code>
<code>kubectl get pods</code>	List all Pods	<code>kubectl get pods</code>
<code>kubectl describe deployment <name></code>	Describe a Deployment	<code>kubectl describe deployment myapp</code>
<code>kubectl scale deployment <name> --replicas=<n></code>	Scale Deployment	<code>kubectl scale deployment myapp --replicas=5</code>
<code>kubectl rollout status deployment <name></code>	Check rollout status	<code>kubectl rollout status deployment myapp</code>

KUBERNETES NOTES

— Page 10 —

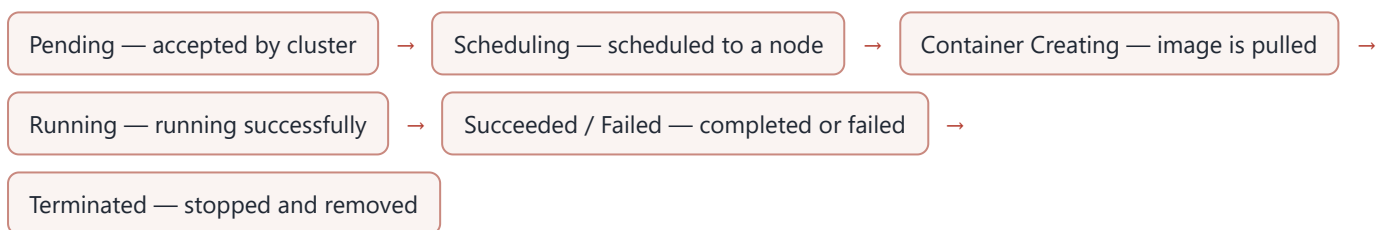
1. ConfigMaps vs Secrets (Quick View)

ConfigMap	Secret
Stores non-sensitive configuration data. Plain text (Base64 not used). Used as environment variables or config files. Use for app settings, URLs, feature flags.	Stores sensitive data. Base64 encoded (not encrypted). Used as environment variables or mounted as files. Use for passwords, tokens, keys, certificates.

2. Resource Requests vs Limits

Requests	Limits
Minimum resources required. Used by scheduler to place Pods. If not set, may lead to poor scheduling.	Maximum resources a container can use. Prevents one container from using all resources. If exceeded, container may be throttled or killed.

3. Pod Lifecycle (In Short)



4. Node Types

- **Master Node (Control Plane):** Manages the cluster, makes global decisions.
- **Worker Node:** Runs the application Pods.
- **etcd Node:** Stores all cluster data (key-value store).

5. Common Labels

- `app` : Name of the application
- `env` : Environment (dev, test, prod)
- `tier` : Tier (frontend, backend)
- `version` : Version of the application
- `team` : Owning team

6. Health Checks (Probes)

- **Liveness Probe:** Is the container running?
- **Readiness Probe:** Is the container ready to serve traffic?
- **Startup Probe:** Is the application started?

7. Useful kubectl Commands

Command	Description
<code>kubectl get pods</code>	List all Pods
<code>kubectl get svc</code>	List all Services
<code>kubectl get nodes</code>	List all Nodes
<code>kubectl describe pod <pod-name></code>	Details of a Pod
<code>kubectl logs <pod-name></code>	Logs of a Pod
<code>kubectl exec -it <pod-name> -- /bin/bash</code>	Shell into Pod